MEDINFERA

Hospital Management SaaS Platform

Project Technical Report • Version 2.0 • 2026

GitHub Organization	github.com/MedInfera
Backend API (Render)	medinfera-backend.onrender.com
Frontend (Vercel)	medinfera-frontend-anuv.vercel.app

Project Team

Name	Role	Domain	Key Responsibilities
Aditya Singh	Backend Lead	Backend	Architecture, JWT auth, hospital & doctor modules, Render deployment
Priyanshi Mishra	Backend Developer	Backend	Prisma schema, ambulance, beds, IPD, lab, medicines, invoices, payouts
Shashwat Srivastava	Database Engineer	Database	PostgreSQL schema design, enums, indexes, triggers, Neon DB
Akshat	Frontend Developer	Frontend	React + Vite UI, role-based dashboards, Tailwind design, Vercel deployment

1. Executive Summary

MedInfera is a production-grade, multi-tenant Hospital Management SaaS (Software as a Service) platform designed to digitize and streamline end-to-end hospital operations. Built by a four-person engineering team, the system spans a React-based frontend, a Node.js/Express REST API backend, and a rigorously designed PostgreSQL relational database hosted on Neon — all deployed on cloud infrastructure (Render for backend, Vercel for frontend).

The platform supports six distinct user roles — Super Admin, Admin, Doctor, Staff, Pharmacist, and Patient — each with tailored dashboards and permission boundaries enforced via JWT-based Role-Based Access Control (RBAC). Every hospital registered on the platform operates as an isolated tenant, ensuring data privacy and operational independence across organizations.

MedInfera covers the complete lifecycle of hospital operations: patient registration, OPD/IPD management, appointment scheduling, pharmacy dispensing, laboratory order tracking, bed and ward management, ambulance dispatch with real-time GPS tracking, billing and invoicing, payroll, and system-wide notifications — all backed by immutable audit logs.

2. Project Overview

2.1 Problem Statement

Healthcare institutions in India and emerging markets continue to rely on fragmented, paper-based workflows or legacy software that lacks integration between clinical, operational, and financial functions. MedInfera addresses this gap by providing a unified, cloud-native platform accessible to multiple hospitals simultaneously through a SaaS model — reducing IT overhead and enabling real-time operational visibility.

2.2 Solution

MedInfera provides a centralized, role-aware platform where every healthcare touchpoint — from a patient walking in to a doctor writing a prescription, a pharmacist dispensing medicines, a lab technician reporting results, and a billing executive generating an invoice — is captured, linked, and traceable. The system is built SaaS-first, meaning each hospital is an independent tenant sharing infrastructure but with fully isolated data.

2.3 Deployment Links

GitHub	https://github.com/MedInfera
Backend API	https://medinfera-backend.onrender.com/api/v1
Frontend	https://medinfera-frontend-anuv.vercel.app

Backend Host	Render (Free Plan — Node.js, auto-deploy from main branch)
Frontend Host	Vercel (auto-deploy from main branch)
Database	Neon PostgreSQL (Serverless, production-compatible)

3. Technology Stack

3.1 Frontend

Technology	Version	Purpose
React	18.2.0	Component-based UI framework
Vite	5.1.0	Build tool and dev server
Tailwind CSS	3.4.1	Utility-first CSS styling
React Router	6.22.0	Client-side routing
Axios	1.6.7	HTTP client for API calls
DM Sans / DM Serif Display	—	Typography (Google Fonts)

3.2 Backend

Technology	Version	Purpose
Node.js	>=18.0.0	JavaScript runtime
Express.js	4.18.3	REST API framework
Prisma ORM	5.10.0	Type-safe database client & migrations
PostgreSQL (Neon)	—	Primary relational database
JWT	9.0.2	Access + Refresh token auth
Socket.io	4.7.4	Real-time ambulance GPS updates
bcryptjs	2.4.3	Password hashing
Joi	17.12.2	Request input validation
Winston	3.12.0	Structured logging
Helmet	7.1.0	HTTP security headers
node-cron	3.0.3	Background job scheduling
morgan	1.10.0	HTTP request logging

3.3 Database

Technology	Purpose
PostgreSQL (Neon Serverless)	Primary database — multi-tenant, ACID-compliant
uuid-ossp extension	UUID primary key generation
pg_trgm extension	Fuzzy full-text search on patient and medicine names

Technology	Purpose
Prisma	Schema management, migrations, query builder
JSONB columns	Flexible storage for hospital settings, job payloads, audit data

4. System Architecture

4.1 Multi-Tenant SaaS Design

MedInfera implements logical multi-tenancy: all hospitals share a single PostgreSQL database instance on Neon, but every table includes a hospital_id foreign key that isolates data per tenant. The SUPER_ADMIN role (hospital_id = NULL) operates at the platform level and can manage all hospitals, while every other user is scoped to their hospital.

This design avoids schema-per-tenant complexity while maintaining strict data isolation. All queries in the backend are scoped by hospital_id, enforced at the middleware and service layer.

4.2 Backend Architecture

The backend follows a modular monolith architecture where each healthcare domain is encapsulated as an independent module under src/modules/. Each module contains its own routes, controllers, services, and validation schemas.

Layer	Component	Responsibility
Entry	src/server.js + src/app.js	Server initialization, middleware registration, route mounting
Middleware	auth.js, rbac.js, validate.js	JWT verification, role checks, Joi input validation
Routing	modules/*/routes.js	Express route definitions per domain module
Business Logic	modules/*/service.js	Core operations, Prisma queries, data transformations
Database	Prisma Client	Type-safe ORM queries against Neon PostgreSQL
Real-time	Socket.io	Ambulance GPS broadcasting to connected clients
Jobs	src/jobs/scheduler.js	node-cron scheduled background tasks
Logging	Winston + morgan	Structured daily-rotated logs, request logging

4.3 Frontend Architecture

The frontend is a Single Page Application (SPA) built with React 18 and Vite. Routing is handled by React Router 6 with role-based protected routes. Each user role (Super Admin, Admin, Doctor, Staff, Pharmacist, Patient) has a dedicated dashboard context. All API calls use Axios with delay-simulated mock services for development, making the frontend functional without a live backend connection during prototyping.

4.4 API Structure

Base URL	https://medinfera-backend.onrender.com/api/v1

All API responses follow a unified envelope format:

Status	Structure
Success	{ "success": true, "message": "Operation successful", "data": { ... } }
Error	{ "success": false, "message": "Error description", "error": { ... } }

5. Database Schema

5.1 Schema Overview

The PostgreSQL schema (v1.0.0, Neon-compatible) defines 35 tables organized across 15 domain sections, along with 28 custom ENUM types and a comprehensive index strategy. Auto-update triggers maintain updated_at timestamps across all mutable tables.

Section	Tables	Domain
Core / Platform	hospitals, users, refresh_tokens, audit_logs	Multi-tenancy, Auth, Audit
Doctor & Schedule	doctor_profiles, doctor_schedules, doctor_leaves	Doctor management
Patients	patient_profiles	Patient records
Wards & Beds	wards, beds	Infrastructure
Appointments	appointments	OPD scheduling
IPD	ipd_admissions, ipd_bed_transfers, ipd_attending_doctors, vital_signs, ipd_notes	Inpatient care
Medicines & Inventory	medicines, medicine_batches, suppliers, purchase_orders, purchase_order_items	Pharmacy stock
Prescriptions & Pharmacy	prescriptions, prescription_items, pharmacy_dispensings, pharmacy_dispensing_items	Drug dispensing
Laboratory	lab_tests, lab_test_panels, lab_panel_tests, lab_orders, lab_order_items	Diagnostics
Ambulance	ambulances, ambulance_dispatches, ambulance_location_trail	Emergency transport
Billing & Payments	invoices, invoice_items, payments	Financials
Payroll & Payouts	staff_payroll, payouts	HR / Compensation
Notifications & Jobs	notifications, background_jobs	Alerts & Automation

5.2 Core Tables

hospitals — Root Tenant Entity

Every hospital on the platform is a row in this table. Key fields: id (UUID PK), name, code (unique short code), slug (URL-safe), subscription_plan (TRIAL / STARTER / PROFESSIONAL / ENTERPRISE), settings (JSONB for hospital-level configuration), and soft-delete via deleted_at.

users — Unified Authentication Table

All roles share a single users table. The hospital_id is NULL only for SUPER_ADMIN, enabling platform-level access. Email uniqueness is enforced per-hospital via a composite unique constraint

(email, hospital_id). Security fields include password_hash (bcrypt), must_reset_password, is_email_verified, and last_login_at.

refresh_tokens

Stores hashed JWT refresh tokens with device context (user_agent, ip_address) and expiry. Revocation is tracked via revoked_at, enabling secure token rotation without database lookups on every request.

audit_logs — Immutable Audit Trail

Captures every sensitive action with old_values and new_values as JSONB, preserving full history for compliance. Cannot be updated or deleted by application logic.

5.3 Clinical Tables

patient_profiles

Extends users with clinical data: date_of_birth, gender (MALE/FEMALE/OTHER), blood_group, allergies and chronic_conditions as TEXT arrays, emergency contact, and insurance details. Patients receive an auto-generated patient_code (e.g., MED-0001) unique per hospital. Full-text GIN index enables fast name search.

appointments

Full appointment lifecycle: SCHEDULED → CONFIRMED → IN_PROGRESS → COMPLETED / CANCELLED / NO_SHOW / RESCHEDULED. Supports OPD token assignment, self-referential rescheduling (rescheduled_from), and both in-person and teleconsult types.

ipd_admissions

Central IPD record linking patient, primary doctor, bed, and ward. Tracks status through ADMITTED → UNDER_TREATMENT → STABLE/CRITICAL → DISCHARGED/TRANSFERRED/DECEASED. Supports multi-doctor care via ipd_attending_doctors and full bed transfer history via ipd_bed_transfers.

vital_signs

Shared across OPD and IPD contexts. Records BP, heart rate, temperature, SpO2, weight, height, BMI, blood glucose, and pain scale (0–10). Linked to either an appointment or an admission depending on context.

5.4 Pharmacy & Lab Tables

medicines + medicine_batches

Medicines carry category (tablet, capsule, syrup, injection, etc.), GST rate, reorder level, and controlled substance flag. Batches enable FIFO dispensing with per-batch expiry tracking. A GIN index on name + generic_name supports fast fuzzy medicine search.

prescriptions + dispensings

Prescriptions are linked to either an appointment (OPD) or an admission (IPD). Items specify dosage, frequency, route, duration, and quantity prescribed vs dispensed. Dispensing records track which pharmacist dispensed which batch.

lab_orders

Supports ROUTINE, URGENT, and STAT priorities. Tracks full workflow: ORDERED → SAMPLE_COLLECTED → IN_PROGRESS → COMPLETED → REPORTED. Each result can flag abnormal values. Panels group related tests for composite ordering.

5.5 Operations Tables

ambulances + ambulance_dispatches + ambulance_location_trail

Ambulances carry type (BASIC, ALS, ICU_MOBILE, MORTUARY, NEONATAL), GPS coordinates, and document expiry dates (registration, insurance, fitness). Dispatches track the full pickup lifecycle with timestamps. The location trail table stores real-time GPS pings (latitude, longitude, speed) broadcast via Socket.io.

invoices + payments

Invoices support OPD, IPD, Lab, Pharmacy, Ambulance, Package, and Miscellaneous types. The financial model includes subtotal, discounts, GST, round-off, and due tracking. Payments support Cash, Card, UPI, NEFT, Cheque, Insurance, and Credit modes.

staff_payroll + payouts

Monthly payroll records per user with basic salary, allowances, and deductions. Payouts cover salary disbursements, advances, reimbursements, bonuses, and deductions. Unique constraint prevents duplicate payroll for the same user-month-year combination.

6. Custom ENUM Types

The schema defines 28 PostgreSQL ENUM types, ensuring data integrity at the database level without relying solely on application-layer validation.

ENUM Name	Values
user_role	SUPER_ADMIN, ADMIN, DOCTOR, RECEPTIONIST, NURSE, PHARMACIST, LAB_TECHNICIAN, BILLING, PATIENT, DRIVER, STAFF
gender	MALE, FEMALE, OTHER
blood_group	A_POSITIVE, A_NEGATIVE, B_POSITIVE, B_NEGATIVE, AB_POSITIVE, AB_NEGATIVE, O_POSITIVE, O_NEGATIVE, UNKNOWN
appointment_type	CONSULTATION, FOLLOW_UP, EMERGENCY, TELECONSULT
appointment_status	SCHEDULED, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED, NO_SHOW, RESCHEDULED
ipd_status	ADMITTED, UNDER_TREATMENT, CRITICAL, STABLE, DISCHARGED, TRANSFERRED, ABSCONDED, DECEASED
bed_type	GENERAL, PRIVATE, SEMI_PRIVATE, ICU, HDU, NICU, PICU, ISOLATION
bed_status	AVAILABLE, OCCUPIED, RESERVED, MAINTENANCE, BLOCKED
ward_type	GENERAL, SURGICAL, PEDIATRIC, MATERNITY, ORTHOPEDIC, CARDIAC, NEUROLOGY, ONCOLOGY, ICU, EMERGENCY, OTHER
medicine_category	TABLET, CAPSULE, SYRUP, INJECTION, DROPS, CREAM, OINTMENT, INHALER, POWDER, PATCH, SUPPOSITORY, OTHER
prescription_status	ACTIVE, PARTIALLY_DISPENSED, FULLY_DISPENSED, CANCELLED, EXPIRED
lab_order_status	ORDERED, SAMPLE_COLLECTED, IN_PROGRESS, COMPLETED, CANCELLED, REJECTED
ambulance_type	BASIC, ADVANCED_LIFE_SUPPORT, ICU_MOBILE, MORTUARY, NEONATAL
ambulance_status	AVAILABLE, DISPATCHED, EN_ROUTE_TO_PICKUP, AT_SCENE, TRANSPORTING, RETURNING, MAINTENANCE, INACTIVE
dispatch_status	REQUESTED, DISPATCHED, ARRIVED_AT_SCENE, PATIENT_PICKED, COMPLETED, CANCELLED
invoice_type	OPD, IPD, LAB, PHARMACY, AMBULANCE, PACKAGE, MISCELLANEOUS
invoice_status	DRAFT, ISSUED, PARTIALLY_PAID, PAID, OVERDUE, CANCELLED, REFUNDED
payment_mode	CASH, CARD, UPI, NETBANKING, CHEQUE, INSURANCE, CREDIT, REFUND
subscription_plan	TRIAL, STARTER, PROFESSIONAL, ENTERPRISE

ENUM Name	Values
notification_type	APPOINTMENT_REMINDER, APPOINTMENT_UPDATE, LAB_RESULT_READY, MEDICINE_LOW_STOCK, MEDICINE_EXPIRY, INVOICE_GENERATED, PAYMENT_RECEIVED, BED_RELEASED, AMBULANCE_UPDATE, GENERAL, SYSTEM_ALERT
job_type	MEDICINE_EXPIRY_CHECK, LOW_STOCK_CHECK, APPOINTMENT_REMINDER, PAYOUT_CALCULATION, BED_AUTO_RELEASE, INVOICE_OVERDUE_CHECK, AUDIT_CLEANUP, PATIENT_REPORT_GENERATION
job_status	PENDING, RUNNING, COMPLETED, FAILED, SKIPPED

7. Indexes & Performance

The schema includes 38 strategically placed indexes covering the most performance-critical query patterns across all modules.

Domain	Index Strategy
users	hospital_id, email, role, phone, composite (hospital_id + role), partial active users
patient_profiles	hospital_id, phone, patient_code, GIN full-text search on first_name + last_name
appointments	hospital_id, (doctor_id + date), patient_id, status, date
ipd_admissions	hospital_id, patient_id, partial on bed_id (active only), status, doctor_id
beds	composite (hospital_id + status), ward_id
medicines	hospital_id, partial stock index (active, low stock), GIN full-text on name + generic_name
medicine_batches	medicine_id, partial expiry (non-zero quantity only)
invoices	hospital_id, patient_id, status, partial due date (unpaid only)
notifications	composite (user_id + is_read), hospital_id
audit_logs	hospital_id, (entity_type + entity_id), user_id, created_at DESC
background_jobs	partial on (status + scheduled_at) for PENDING/FAILED only
ambulances	composite (hospital_id + status)

7.1 Auto-Update Triggers

A PL/pgSQL trigger function (trigger_set_updated_at) is applied to 22 tables using a dynamic DO block. This ensures updated_at is always accurate without requiring application-layer tracking — a critical requirement for audit consistency and cache invalidation.

8. Security Architecture

Security Layer	Implementation
Authentication	JWT access tokens (15 min expiry) + refresh tokens (7 days) stored hashed in DB
Password Security	bcrypt hashing with 12 rounds (configurable via env)
Role-Based Access Control	Middleware-enforced RBAC on every protected route
Input Validation	Joi schema validation on all request bodies before processing
HTTP Security	Helmet middleware for HSTS, X-Frame-Options, CSP headers
Rate Limiting	express-rate-limit: 100 requests per 15-minute window (configurable)
Token Rotation	Refresh tokens are hashed and carry revocation timestamps
Multi-Tenant Isolation	All queries scoped by hospital_id at service layer
Audit Trail	Immutable audit_logs table captures all sensitive operations with IP + user agent
SQL Injection	Prevented via Prisma ORM parameterized queries — no raw SQL in routes

8.1 Demo Credentials

The frontend includes role-specific demo accounts for evaluation:

Role	Email	Password	Route
Super Admin	superadmin@medinfera.com	super@2026	/superadmin
Admin	admin@medinfera.com	admin@2026	/admin
Doctor	doctor@medinfera.com	doctor@2026	/doctor
Staff	staff@medinfera.com	staff@2026	/staff
Pharmacist	pharmacist@medinfera.com	pharma@2026	/pharmacist
Patient	patient@medinfera.com	patient@2026	/patient

9. Backend Modules

The backend is organized into 14 domain modules under src/modules/, each with its own routing, controller, and service layer:

Module	Key API Capabilities
auth	Login, refresh, logout, password reset, token management
users	User CRUD, profile management, role-based access, activity tracking
hospitals	Hospital registration, config, subscription management, tenant setup
doctors	Doctor profiles, schedules, leave management, availability queries
patients	Patient registration, search, medical history, profile updates
appointments	Book, confirm, reschedule, cancel, OPD token assignment
ipd	Admissions, discharge, bed transfers, attending doctors, nursing notes, vitals
beds	Ward management, bed status, allocation tracking
medicines	Inventory CRUD, batch management, low-stock alerts
prescriptions	Digital prescriptions, dispensing workflow, status tracking
lab	Test catalog, panel management, order creation, result entry
ambulance	Fleet management, dispatch, real-time GPS via Socket.io
invoices	Invoice generation, GST calculation, payment recording
payouts	Monthly payroll, salary disbursement, advance tracking, payout records
notifications	System alert delivery, read status, type-filtered queries

10. Deployment & Infrastructure

10.1 Backend — Render

Configuration	Value
Platform	Render (Free Plan, Node.js environment)
Build Command	npm install && npx prisma generate && npx prisma migrate deploy
Start Command	node src/server.js
Auto Deploy	Enabled — triggers on every push to main branch
JWT Access Token TTL	15 minutes
JWT Refresh Token TTL	7 days
Rate Limit Window	15 minutes (900,000 ms)
Rate Limit Max Requests	100 per window
bcrypt Rounds	12
CORS	Wildcard (*) — to be restricted to production frontend domain

10.2 Frontend — Vercel

Configuration	Value
Platform	Vercel
Build Tool	Vite 5.1.0
Framework	React 18
Dev Port	5173 (localhost)
CSS Framework	Tailwind CSS 3.4.1
Font Stack	DM Sans, DM Serif Display, DM Mono (Google Fonts)

10.3 Database — Neon

Configuration	Value
Provider	Neon (Serverless PostgreSQL)
Extensions	uuid-ossp (UUID generation), pg_trgm (fuzzy search)
Migrations	Prisma Migrate — applied automatically on each Render deploy
Timezone	Asia/Kolkata (IST)
Currency	INR (configurable per hospital)

11. Background Jobs & Automation

MedInfera implements a cron-based background job scheduler (node-cron) for automated operational tasks. Jobs are tracked in the background_jobs table with status, payload, result, error messages, and retry logic (up to 3 retries per job).

Job Type	Trigger	Description
MEDICINE_EXPIRY_CHECK	Daily	Identifies medicine batches approaching or past expiry and triggers MEDICINE_EXPIRY notifications
LOW_STOCK_CHECK	Daily	Compares current_stock against reorder_level and sends MEDICINE_LOW_STOCK alerts
APPOINTMENT_REMINDER	Hourly	Sends APPOINTMENT_REMINDER notifications to patients for upcoming appointments
PAYOUT_CALCULATION	Monthly	Processes monthly payroll calculation for all active staff
BED_AUTO_RELEASE	Daily	Releases beds from RESERVED status after discharge is confirmed
INVOICE_OVERDUE_CHECK	Daily	Marks ISSUED invoices past due_date as OVERDUE and notifies billing staff
AUDIT_CLEANUP	Weekly	Archives or purges old audit log entries per retention policy
PATIENT_REPORT_GENERATION	On-demand	Generates periodic patient summary reports for admins

12. Real-Time Features

12.1 Ambulance GPS Tracking

MedInfera integrates Socket.io for real-time ambulance location broadcasting. When an ambulance driver updates their GPS coordinates, the server broadcasts the location to all connected clients subscribed to that dispatch. The ambulance_location_trail table persists the complete route history (latitude, longitude, speed_kmh, recorded_at) for post-trip analysis.

Ambulance status transitions (AVAILABLE → DISPATCHED → EN_ROUTE_TO_PICKUP → AT_SCENE → TRANSPORTING → RETURNING) are also broadcast in real-time to dispatch operators and administrators.

12.2 Notification System

The notifications module delivers typed, entity-linked notifications to users. Notifications carry entity_type and entity_id references so the frontend can deep-link to the relevant record. The is_read flag and read_at timestamp enable unread badge counts and read-receipt tracking.

13. Key Design Decisions

Decision	Rationale
Single shared database with hospital_id isolation vs. schema-per-tenant	Simpler operations, easier migrations, and lower infrastructure cost at current scale. hospital_id enforcement at service layer provides sufficient isolation.
Prisma ORM over raw SQL	Type-safe queries, automatic migration management, and schema introspection improve developer velocity and reduce SQL injection risk.
JWT with refresh tokens over session-based auth	Stateless access tokens scale horizontally without session stores. Refresh token rotation in the database enables revocation without distributed state.
PostgreSQL ENUM types for status fields	Enforces valid state transitions at the database level, preventing invalid data even if application validation is bypassed.
TEXT[] for allergies and chronic_conditions	Simpler than a separate junction table for list-type medical data that doesn't require independent querying or complex relationships.
JSONB for hospital settings and audit values	Provides flexible, schema-less storage for configuration and change history without altering table structure as requirements evolve.
GIN indexes for full-text patient and medicine search	pg_trgm GIN indexes enable fast fuzzy search (LIKE '%term%') on large name fields — critical for clinical workflows requiring quick patient and drug lookup.
Separate patient_profiles from users	Patients may or may not have login accounts. Decoupling clinical data from auth data allows walk-in patient registration without creating a user account.
medicine_batches for FIFO dispensing	Tracks per-batch expiry and quantity independently, enabling the pharmacy to dispense oldest batches first and flag approaching expiries automatically.

14. Future Scope

Area	Planned Enhancement
API Documentation	Swagger/OpenAPI 3.0 specification for all endpoints
Testing	Jest unit tests and Supertest integration tests for all modules
Caching	Redis layer for appointment availability, medicine catalog, and dashboard KPIs
Advanced RBAC	Fine-grained permission management beyond role-level access control
File Storage	AWS S3 / Cloudflare R2 for doctor signatures, patient documents, and lab reports
SMS/Email Notifications	Twilio/SendGrid integration for patient appointment reminders
Insurance Integration	API integration with health insurance providers for claim processing
Analytics Dashboard	Aggregated KPIs for hospital admins: occupancy rates, revenue trends, lab TAT
Mobile App	React Native patient-facing app for appointment booking and report access
Microservices Migration	Decompose high-traffic modules (appointments, lab) for independent scaling

15. Conclusion

MedInfera represents a comprehensive, production-ready Hospital Management SaaS platform built with modern engineering practices. The system's multi-tenant architecture, modular backend, type-safe ORM, and comprehensive database design provide a solid foundation for real-world healthcare digitization at scale.

The combination of a well-normalized PostgreSQL schema with strategic indexing, immutable audit trails, real-time capabilities via Socket.io, and automated background jobs positions MedInfera as a complete solution for hospital operations management — from patient walk-in to discharge, from prescription to dispensing, and from lab order to result reporting.

The project demonstrates strong full-stack engineering discipline: clean separation of concerns in the backend, a consistent API design, robust security layers, and a role-aware frontend — all deployed on cloud infrastructure with automated CI/CD pipelines.

MedInfera — Built with care for healthcare • github.com/MedInfera